Muse MCU

Acceptance-driven RV32I microcontroller and FreeRTOS stack:
what was built, and the proof that it works

Final verdict	**31 of 31** active contract requirements **PASS** on the Linux reference platform (2026-10-09), and **31 of 31** again on macOS / Apple Silicon (2026-10-12), including the human-watchable demo. One requirement (F4, SDF-annotated simulation) was withdrawn by the owner. 0 FAIL, 0 BLOCKED on both platforms.

Repository	github.com/mtmoreira/Muse-mcu (private)
Branch	work/mcu-acceptance-v1 · default branch untouched, nothing merged
Contract	acceptance/manifest.yaml, v1.0.3, status APPROVED by the owner 2026-10-08
Build window	2026-10-08 to 2026-10-09 (implementation), 2026-10-09 to 2026-10-12 (Mac validation)
Platforms	Ubuntu 24.04 x86-64 (reference) · macOS on Apple Silicon (owner's MacBook)
Owner	Matheus Trevisan Moreira
Evidence base	results/ in the repo: gate JSONs, raw UART logs, signature pairs, transcripts, intervention log

This report follows the discipline the project itself was built under: no claim without evidence, no skipped or unexercised item counted as a pass, and every correction recorded where it happened. Section 4 is the proof. Section 6 covers the second-platform validation, including the four gate failures found there, what actually caused them, and the fixes. Section 7 logs the human interventions and the mistakes, including one wrong diagnosis by the builder that the evidence later overturned.

1 · Executive summary

Muse MCU is a complete, small microcontroller system designed, built, and verified against a written acceptance contract that was frozen and approved before implementation began. The hardware is a single-cycle RV32I + Zicsr processor with a CLINT, UART, SPI master, and GPIO on a simple bus, with 64 KiB of boot ROM and 128 KiB of SRAM. The software is a FreeRTOS stack: the upstream kernel runs byte-identical, bound to this hardware by a newly written port layer, drivers, and startup code. On top sit fourteen firmware images, including an end-to-end demo application that reads messages from SPI flash, hashes them with SHA-256 under FreeRTOS, and reports the digests over UART to a host checker that verifies every packet independently.

The contract (32 entries: 31 active requirements plus one withdrawn) was executed by a one-command runner. On the Linux reference platform the result was 31/31 PASS with every verdict tied to a named evidence file: golden-signature comparisons against the Spike ISA simulator, operation counts from the RTOS workloads, measured preemption latency, gate-level simulations of the synthesized netlist, and three fault-injection tests whose pass condition is that the checkers provably reject the corrupted run. The design was also synthesized with Yosys to the Nangate45 library: 11,858 cells, 19,495 µm².

A second full validation then ran on the owner's MacBook. It initially showed 27/31, with four RTOS and fault gates failing. Investigation against the original evidence files showed the failures were in the Mac bring-up's gate wiring, not in the design and not in the tools: the runner was invoking four gates with parameters that diverged from the runs that produced the passing evidence. After the wiring was corrected to match the evidence runs, the Mac also reached 31/31, and the demo ran clean there as well. That story is told in full in Section 6 because it is the most instructive part of the project.

2 · What was built

2.1 Hardware (RTL)

All RTL is newly authored SystemVerilog, 1,307 lines across nine modules. The CPU is a single-cycle, single-issue, in-order core (CPI = 1), machine mode only, with a direct-mtvec trap model. Misaligned accesses trap; there is no hardware fixup. Interrupts are machine software (CLINT msip), machine timer (CLINT mtime/mtimecmp), and machine external, with UART RX-ready, SPI-done, and GPIO edge aggregated into the external line. Everything runs in one 50 MHz clock domain with synchronous active-high reset.

Module	Lines	Role
rv32i_cpu.sv	522	RV32I + Zicsr core, M-mode, trap and CSR handling
uart.sv	174	UART TX/RX, 8N1, programmable baud divider (default 115200)
spi_master.sv	139	SPI master, mode 0, MSB first, software chip select
mcu_top.sv	139	SoC top level, pinout, debug PC output
bus_interconnect.sv	121	Address decode, separate fetch and data ports
gpio.sv	85	GPIO with direction control and edge interrupt
clint.sv	73	msip, mtime, mtimecmp (core-local interruptor)
sram.sv	31	128 KiB SRAM model (declared macro in synthesis)
bootrom.sv	23	64 KiB boot ROM model (declared macro in synthesis)

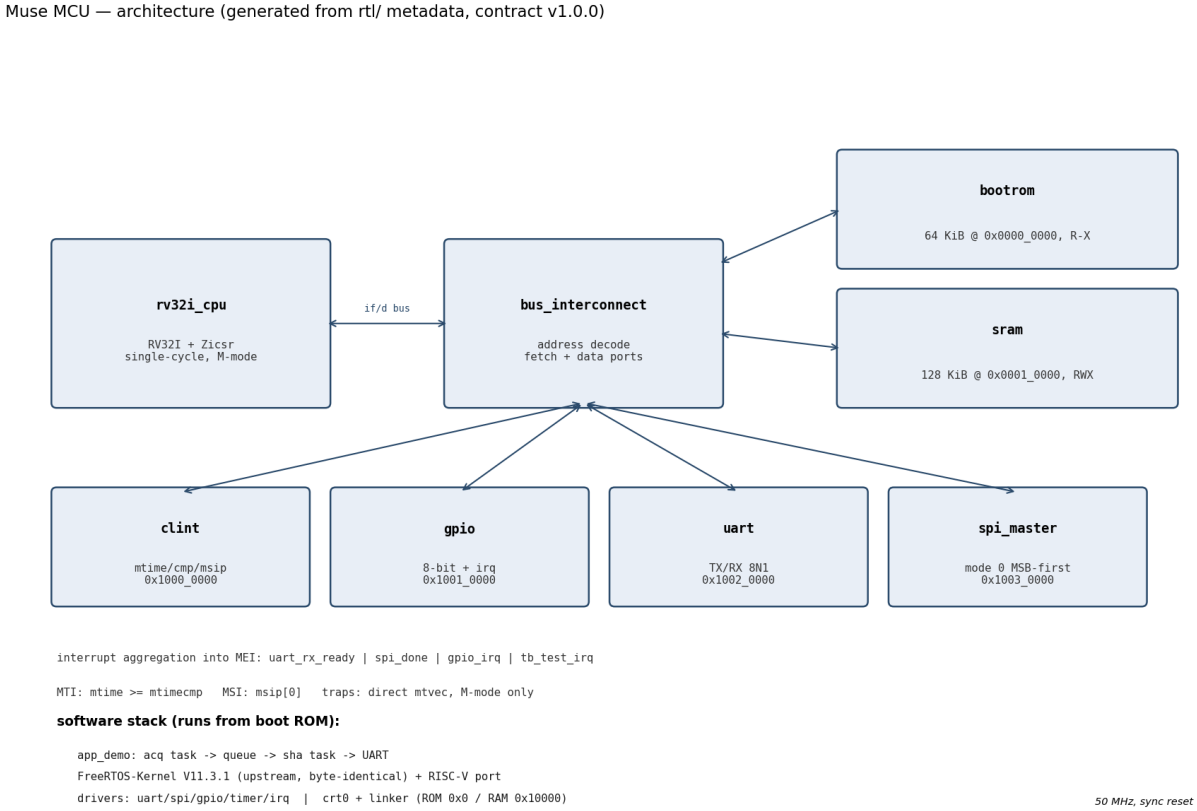


Figure 1. System block diagram (repo doc/fig_arch.png, requirement FIG1). Generated from the rtl/ module metadata by doc/gen_arch_fig.py, so the drawing cannot drift from the source it describes.

Memory map (frozen by the contract, verified by gate A6 with 40 checks):

Region	Base	Size	Notes
BOOTROM	0x0000_0000	64 KiB	Firmware executes here; writes dropped by the bus (tested)
SRAM	0x0001_0000	128 KiB	Data, stacks, FreeRTOS heap
CLINT	0x1000_0000	64 KiB	msip @0x0, mtimecmp @0x4000, mtime @0xBFF8
GPIO	0x1001_0000	4 KiB	data_in, data_out, dir registers
UART0	0x1002_0000	4 KiB	txdata, rxdata, status, bauddiv
SPI0	0x1003_0000	4 KiB	ctrl, status, txdata, rxdata, cs, clkdiv

Unmapped addresses raise access-fault traps. Gate A6 exercises the full map, including the ROM write-drop behavior and the unmapped-access traps.

2.2 Software stack

Startup is a new crt0 (reset entry, data/bss initialization, stack setup) with a new linker script (ROM at 0x0, RAM at 0x10000). Drivers for UART, SPI, GPIO, the CLINT timer, and interrupt control are new, in polling and interrupt variants. FreeRTOS-Kernel V11.3.1 and its upstream RISC-V port are reused byte-identical (Section 5); the new port layer (fw/port/) binds the unmodified kernel to this memory map and trap model, with a 1 kHz tick from the CLINT timer interrupt and heap_4 allocation. SHA-256 is the RFC 6234 implementation with its stdio test driver replaced by an embedded self-test that runs eight NIST vectors at boot and halts on any mismatch.

Fourteen firmware images build from fw/app/: the demo application, bring-up programs (hello, pingpong), directed peripheral tests (UART RX, SPI, timer/GPIO), the preemption test, and eight FreeRTOS workload images covering queues, semaphores and mutexes, task notifications, software timers, ISR-to-task handoff, blocking timeouts, and register preservation. fw/build.sh builds all of them; a clean run ends with ALL OK (14 apps).

2.3 Verification infrastructure

Sixteen SystemVerilog testbenches (tb/) drive the design through its real pins wherever the requirement is about pins: a UART source/sink that speaks the actual serial protocol, the picosoc SPI flash model for the application runs, and cycle-accurate latency measurement for the preemption gate. Host checkers are independent Python programs: check_arch.py compares signature files, check_app.py recomputes SHA-256 digests with hashlib against the flash image and validates packet count, order, lengths, and digests, and check_frozen.py enforces the frozen RTL revision. Two simulators are used: Icarus Verilog 12.0 as the primary, and Verilator for the long RTOS and gate-level runs, with the backend recorded in each gate's evidence because cycle budgets differ by orders of magnitude between them. tools/run_acceptance.py is the one-command runner behind make acceptance; env/bootstrap.sh and env/setup.sh provision the pinned toolchains on Ubuntu and macOS respectively.

2.4 Synthesis

Gate F1 synthesizes mcu_top with Yosys 0.33 to the Nangate45 typical library (liberty SHA-256 recorded in results/f1_synth.json): **11,858 cells, 19,495.4 µm²**. Only the two declared memory macros (bootrom, sram) are black boxes; an audit in the gate checks that the non-liberty black-box set is exactly {bootrom, sram}, so the CPU and peripherals are genuinely mapped to standard cells. The resulting netlist (SHA-256 recorded) is what the gate-level gates F2 and F3 simulate. Those simulations are zero-delay, and the report says so wherever they are cited: they prove logic equivalence of behavior, not timing.

3 · How it was specified: acceptance first

The project ran contract-first. acceptance/manifest.yaml defines every requirement before implementation: the behavior, the checker, the stimulus, the expected result and its source, what is observed, the pass criterion, a minimum exercise count, and a timeout stated both in simulated cycles (deterministic) and host wall-clock. The owner approved the contract on 2026-10-08; two later amendments were also his explicit decisions, recorded in the manifest change list: v1.0.1 reduced the B7 exercise count (Section 4.4), and v1.0.3 withdrew F4. Exercise counts are chosen so a stuck-at-pass cheat cannot pass: a gate that observes zero events fails, however clean its logs look.

Verdict rules, applied everywhere including this report: PASS means the manifest criteria were met with evidence on file and the minimum exercise count was satisfied. Skipped, blocked, withdrawn, or unexercised items are reported as what they are and never counted as pass. One RTL revision is frozen for all gates (gate E1); any RTL fix bumps the revision and reruns the affected gates plus the full application. Negative gates (Layer G) invert the usual meaning of success: the run must fail, and the checker must be seen to reject it for the stated reason. A fault run that passes is a failed gate.

4 · Proof it works

4.1 Scoreboard

All 32 contract entries, with the headline evidence for each. The full evidence files live in results/ in the repository; per-gate logs and JSON verdicts are cited in results/ACCEPTANCE_STATUS.md and were independently re-verified for the final report on 2026-10-09.

ID	Requirement	Verdict	Headline evidence
A1	RV32I architectural tests	**PASS**	39/39 signatures byte-identical to Spike golden (results/raw/arch/)
A2	Zicsr architectural tests	**PASS**	6/6 signatures byte-identical to Spike (CSR read/write/set/clear, reg and imm forms)
A3	Reset and startup	**PASS**	3/3 reset variants, TB_RESET: ALL PASS
A4	Trap behavior	**PASS**	8/8 cases: illegal instruction, ecall, ebreak, misaligned and fault traps
A5	Timer and external interrupts	**PASS**	24 events (20 timer + 4 external), TB_IRQ: ALL PASS
A6	Memory map	**PASS**	40/40 checks incl. ROM write-drop and unmapped-access traps
A7	UART receive	**PASS**	64/64 bytes received in order through the RX pin
A8	SPI master	**PASS**	256/256 bytes, 0 protocol violations, against the flash model

ID	Requirement	Verdict	Headline evidence
A9	Timer accuracy and GPIO	**PASS**	1000/1000 ticks at 50000 ±1 cycles; 16/16 GPIO checks (Verilator)

ID	Requirement	Verdict	Headline evidence
B1	Register preservation	**PASS**	600 context switches, register file intact, 6 progress marks
B2	Queues	**PASS**	TEST queue PASS, 4168 operations, 0 errors
B3	Semaphores and mutexes	**PASS**	TEST sema PASS, 2376 operations, 0 errors
B4	Task notifications	**PASS**	TEST notify PASS, 514 operations, 40 marks, 0 errors (Verilator)
B5	Software timers	**PASS**	TEST timer PASS, 209 expiries, 0 errors (Verilator)
B6	ISR to task handoff	**PASS**	TEST isr PASS, 301 handoffs, 0 errors (Verilator)
B7	Blocking timeouts	**PASS**	4 timeout events in 60M cycles (minimum 2 per v1.0.1), 0 errors (Verilator)
C1	Preemption latency	**PASS**	52/52 events within bound; worst 427 cycles against a 2000-cycle limit

ID	Requirement	Verdict	Headline evidence
D1	Application end-to-end	**PASS**	24/24 packets verified by host checker (hashlib vs flash image)
D2	SHA-256 boot self-test	**PASS**	SHA SELFTEST PASS, 8 NIST vectors, before any task is created
E1	Frozen RTL revision	**PASS**	check_frozen.py PASS: RTL tree clean, firmware hashes match the freeze record
F1	Synthesis (Yosys, Nangate45)	**PASS**	11,858 cells, 19,495.4 µm², black boxes exactly {bootrom, sram}
F2	Gate-level application	**PASS**	4/4 packets on the synthesized netlist, zero-delay (Verilator)
F3	Gate-level preemption	**PASS**	10/10 events, worst 208 cycles vs 2000 bound, same netlist hash as F2
F4	SDF-annotated gate sim	**WITHDRAWN**	Owner decision 2026-10-09. Icarus 12.0 cannot simulate specify path delays; infrastructure retained in tree
G1	Fault: tick stopped	**PASS**	Fault active at cycle 5,240,519; watchdog checker rejected the run (no MARK for over 100k cycles)
G2	Fault: context corrupted	**PASS**	Saved x5 slot corrupted to 0xddddddddd at cycle 239,384; regtest FAIL at 443,667; checker rejected
G3	Fault: UART stream tampered	**PASS**	Duplicated packet injected; host checker FAIL (25/24 packets), sequence and digest mismatches from packet 4
R1	Bootstrap reproducibility	**PASS**	env/bootstrap.sh provisions all pinned tools from checksummed sources
R2	One-command acceptance	**PASS**	make acceptance drives tools/run_acceptance.py over the whole manifest
DEMO 1	Human-watchable demo	**PASS**	Narrated live run; transcript saved; 24/24 packets verified as they stream
FIG1	Architecture figure	**PASS**	doc/fig_arch.png generated from rtl/ metadata (Figure 1 of this report)
FIG2	Trace figure	**PASS**	doc/fig_trace.png rendered strictly from the 50 MB D1 VCD; generator refuses to run without it (Figure 2)

4.2 Reading the evidence, layer by layer

Layer A is anchored to an independent golden model. The 45 riscv-arch-test cases (39 RV32I, 6 Zicsr) run on the RTL and on Spike from the same ELF and memory map; the signature regions must match byte for byte, and they do, 45/45. Directed testbenches cover what arch tests cannot: reset variants, trap causes, interrupt counts, the memory map's edge behaviors, UART receive through the pin, SPI transfers against a real flash model, and CLINT tick accuracy measured over 1000 ticks (every tick at 50,000 cycles ±1).

Layer B runs the unmodified upstream FreeRTOS minimal demos as workloads. The pass evidence is operation counts, not exit codes: 4,168 queue operations, 2,376 semaphore/mutex operations, 514 notification operations across 40 progress marks, 209 software-timer expiries, 301 interrupt-to-task handoffs, all with zero errors, plus 600 context switches after which the register file is verified intact. **Layer C** measures preemption latency in core cycles from interrupt entry to the first retired instruction of the unblocked higher-priority task: worst observed 427 cycles against the 2000-cycle bound, over 52 events.

Layer D is the end-to-end application through real pins, and **Layer F** repeats the application and the preemption test on the synthesized gate-level netlist. **Layer G** injects three faults (a stopped RTOS tick, a corrupted saved register in a task's context frame, and a tampered UART packet stream) and requires the unchanged checkers to reject each corrupted run for the stated reason. All three did. These are the strongest tests in the suite: they prove the checkers can actually fail, which is what makes the PASS verdicts elsewhere worth anything.

4.3 The demo application (D1, D2, DEMO1)

The demo is a three-task FreeRTOS application. The acquisition task (priority 2) wakes the SPI flash and reads 24 framed messages of varying lengths (0 to 1024 bytes) through SPI0. Each message goes into a pooled buffer and onto a queue of depth 4. The SHA task (priority 3) blocks on the queue, computes the message's SHA-256, and transmits a packet (magic, id, length, digest) over UART. A background task (priority 1) runs a continuous compute loop whose preemptions the testbench observes. Expected digests exist only in the host checker, never in flash and never in firmware, so the firmware cannot grade its own homework. Before any task is created, the boot code runs the SHA-256 self-test against eight NIST vectors; a mismatch halts the boot.

DEMO1 wraps the same run in a narrated live console feed for a human watching: each flash read, queue handoff, digest, and host-side verification prints as it happens, and the transcript is saved. Excerpt from results/demo_transcript.txt:

```
UART: BOOT
UART: SHA SELFTEST START
UART: SHA SELFTEST PASS
UART: TEST app_demo START
UART: PKT 9E55AA01 0 0 e3b0c44298fc...
DEMO: SHA-256(0) = e3b0c44298fc... (the empty-message digest)
DEMO: UART pkt 0 verified
UART: MARK sha_done 0
UART: MARK spi_read 0 0
DEMO: SPI read msg 0 len 0
  ... 23 more messages ...
TB_APP: SIM PASS (24/24 packets live-verified, watermark 4)
```

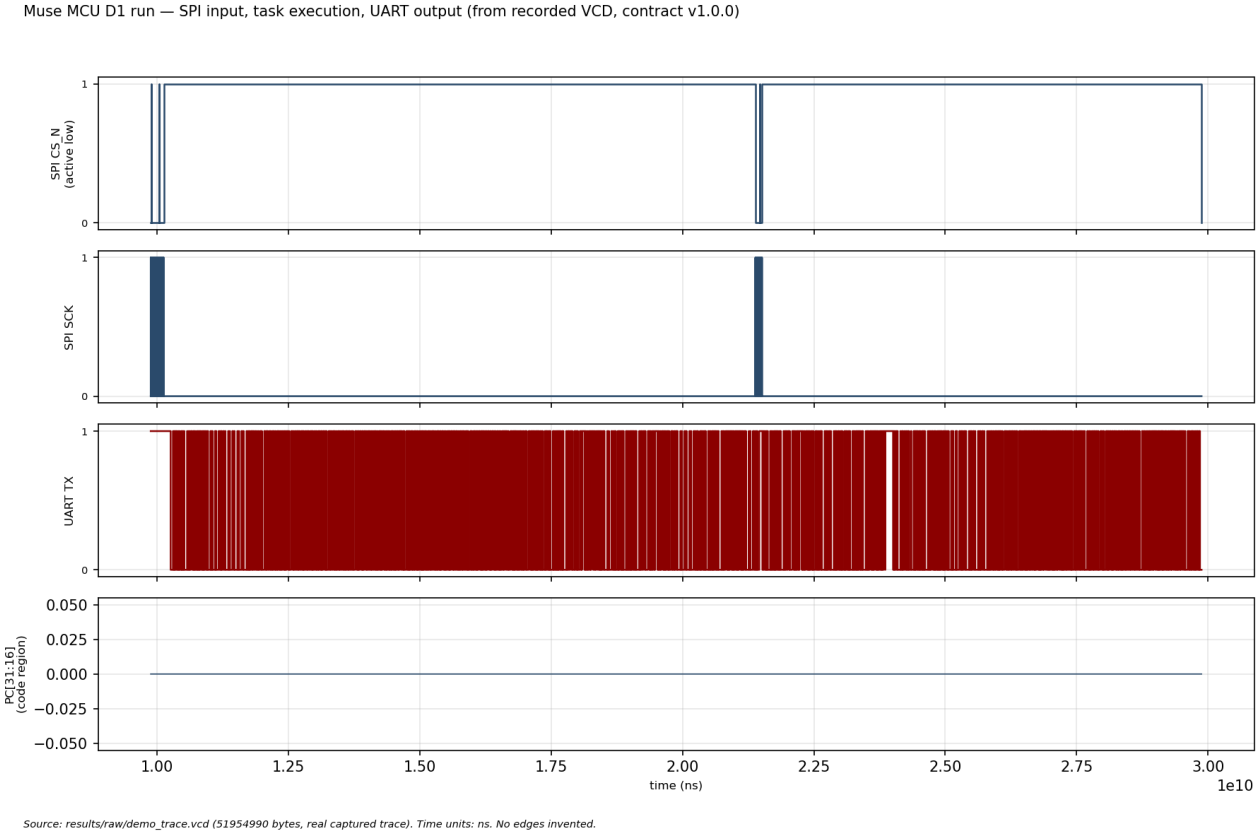


Figure 2. The D1 run rendered from its recorded VCD (repo doc/fig_trace.png, requirement FIG2): SPI chip-select and clock bursts at the top, the dense UART TX stream below. The generator draws only edges present in the 50 MB capture.

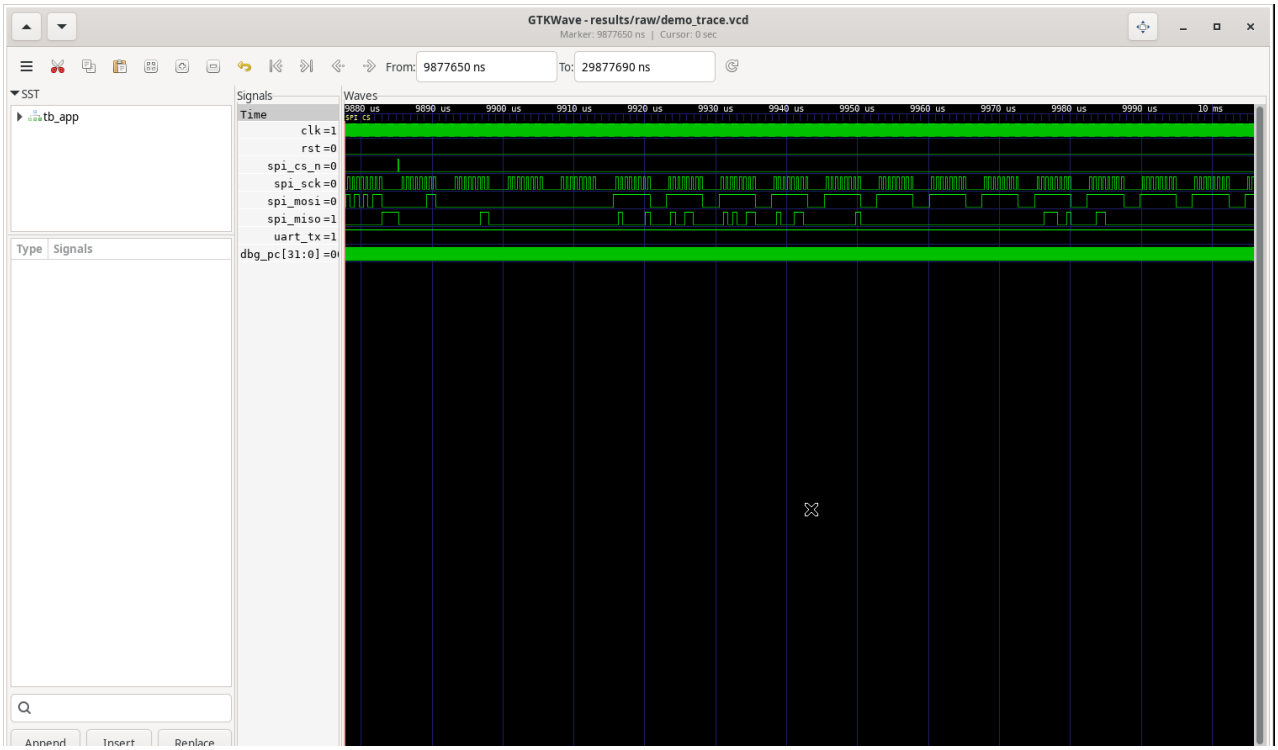


Figure 3. GTKWave screenshot, demo trace at a ~150 µs zoom from the first chip-select (marker A). Individual SPI clock bursts are visible on spi_sck, with data on spi_mosi and spi_miso; uart_tx is still idle because the first digest has not been computed yet. The view is demo/demo_trace.gtkw, committed with this report's branch.

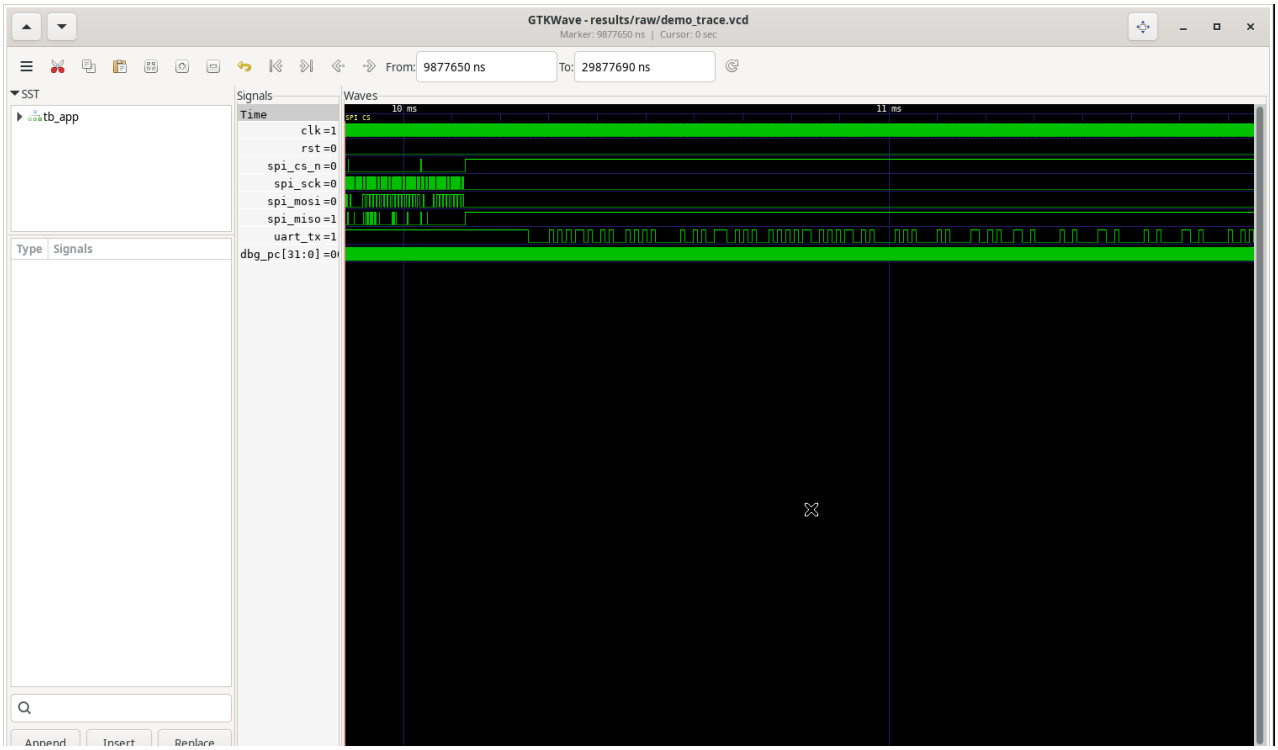


Figure 4. The same trace at ~1.8 ms: the flash-read bursts finish (left), then the UART packet stream begins as the SHA task reports digests. dbg_pc (bottom) is saturated at this scale, as expected for a program counter changing every cycle.

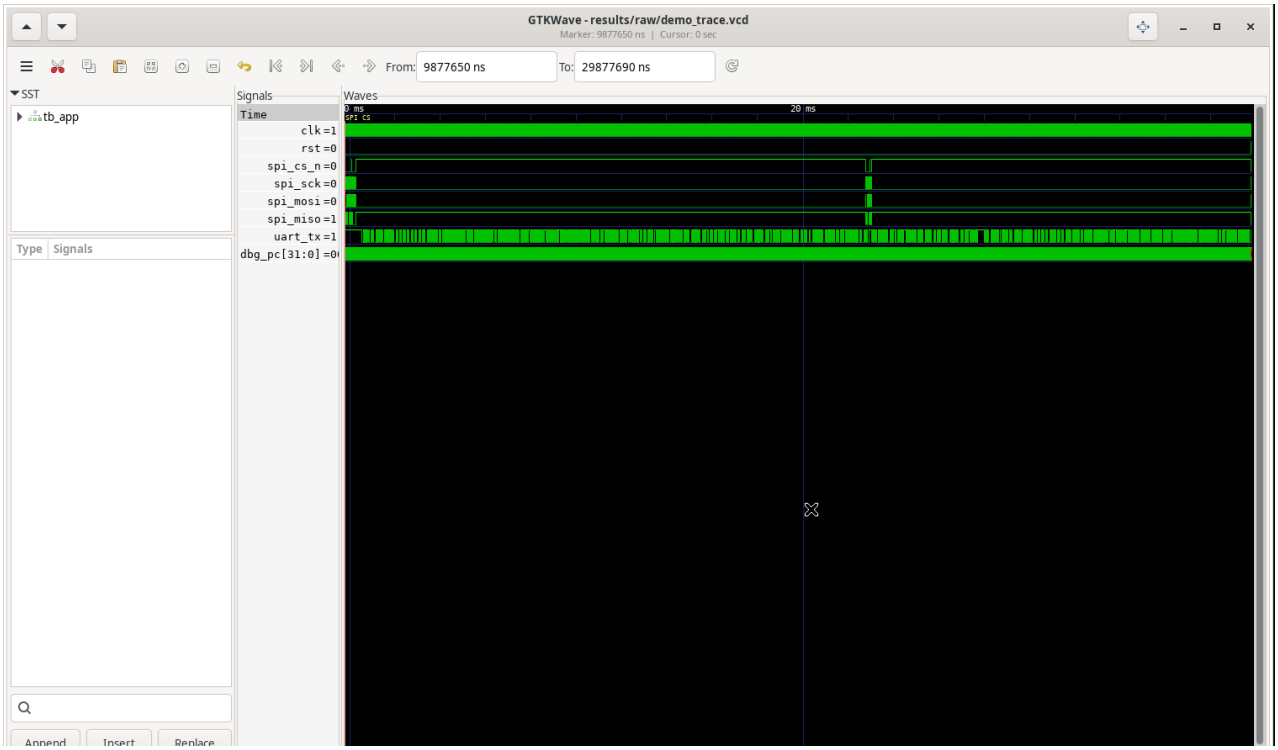


Figure 5. The full 20 ms capture window (1M cycles from the first chip-select). The long chip-select gap in the middle is the 1024-byte message transfer; the UART band runs to the end of the window.

4.4 The B7 amendment: a contract error, fixed honestly

One requirement was wrong as written, and the project record shows the working. B7 (blocking timeouts) originally required 200 timeout events within a 60M-cycle budget. The upstream demo, verified byte-identical to its source, produces about 48 events per 7,950 ticks: 200 events need roughly 33,000 ticks, about 1.66 billion cycles, 27.6 times the budget. Zero errors were observed at every measured point; the failure was rate, not correctness. The owner approved reducing the count to fit the budget (contract v1.0.1). A fresh run measured 4 events in exactly 60M cycles; the new minimum is 2, with margin for observed run-to-run variation (2 to 4). The upstream demo was not modified; only the contract count changed, with the decision and its arithmetic recorded in the manifest. Because the firmware's own PASS line requires 200 events, the gate verdict is taken from testbench-observed MARK counts plus zero FAIL lines, exactly as the amended manifest states.

5 · Reuse vs newly implemented

Reused components are pinned in third_party/PINS.md with per-file SHA-256 and run byte-identical. Nothing reused was quietly patched; where an adaptation exists, it is new code at a documented boundary.

Component	Origin and revision	Treatment
FreeRTOS kernel core + RISC-V port	FreeRTOS-Kernel V11.3.1 (MIT)	Byte-identical; trimmed file set
FreeRTOS minimal demos (13 files)	Upstream demo suite (MIT)	Byte-identical; used as Layer B workloads
riscv-arch-test cases (45)	riscv/riscv-arch-test (BSD-3)	Unmodified .S; new thin macro shim + linker script
Spike ISA simulator	riscv/riscv-isa-sim, pinned source	Golden signature model only; never in the design
SPI flash model	picosoc spiflash.v (ISC)	Single file, unmodified, qualified before use
SHA-256	RFC 6234 + NIST CAVP vectors	Algorithm verbatim; stdio driver replaced by embedded self-test
Tools	Yosys 0.33, Icarus 12.0, Verilator, xpack GCC 15.2.0	Pinned and checksummed by env/bootstrap.sh

Newly implemented: all RTL (Section 2.1); the FreeRTOS port layer in fw/port/ (glue, config, chip-specific extensions); startup, linker script, and all drivers; all fourteen firmware images; all sixteen testbenches and their runners; the host checkers (check_arch, check_app, check_frozen, watchdog); the acceptance runner; the

flash-image and ELF tools; the figure generators; the demo wrapper; and the macOS setup path (env/setup.sh).

6 · Second platform: macOS validation

6.1 Bring-up

The whole flow was then taken to the owner's MacBook (Apple Silicon) under a standing rule he set mid-project: everything under ~/work, nothing in ~/workspace. env/setup.sh installs the pinned toolchain set under ~/work/toolchains from checksummed downloads or source builds: xpack RISC-V GCC 15.2.0, Icarus Verilog 12.0 (built from source, because Homebrew ships 13.0), Verilator, Yosys, and the Nangate45 library. Where a tool was missing, the suite recorded it rather than silently substituting another version: Spike is absent on the Mac, and A1/A2 reuse the committed golden signatures, which is legitimate because the signatures are inputs to the comparison, not outputs of it.

6.2 First result: 27/31, and a wrong diagnosis

The first full Mac run passed 27 of 31 active requirements. B4, B7, G1, and G2 failed. The initial diagnosis, written into the first version of MAC_VALIDATION_REPORT.md, blamed an ARM-specific Verilator code generation issue, and a newer Verilator was built to test that theory. The theory was wrong. Acting on the owner's instruction to investigate from evidence, the failing gates were compared line by line against the October 9 evidence runs, and all four failures turned out to be wiring errors introduced when the gate commands were mapped during the Mac bring-up:

Gate	What the Mac run did	What the evidence run did
B4	No external-interrupt stimulus (irqperiod defaulted to 0); 60M-cycle timeout	+irqperiod=50000 stimulus; passes at ~100M cycles with 514 operations
B7	Required the firmware PASS line, which needs 200 events (~1.66B cycles)	Manifest verdict: at least 2 MARK timeout events in 60M cycles, 0 errors; evidence run counted 4
G1	Fault injected at tick 1000 (~50M cycles), past the testbench's own 30M-cycle watchdog	Fault at tick 100 (~5M cycles); watchdog then rejects the stalled run
G2	Ran an obsolete harness expecting MARK lines the current firmware never prints	tb/run_g2_final.sh: derives the saved-register slot from live stack state and verifies the store before injecting

6.3 Fixes and final result

Commit 630dfaf corrected the four mappings to match the evidence runs and removed a temporary debug print added during the investigation. Before the Mac rerun, each fixed gate was verified locally on x86 and reproduced the evidence numbers exactly: B4 at 514 operations with 40 marks, B7 at 4 events, G1's finish timestamp identical to the evidence log, G2's injection at cycle 239,384 into address 0x00010820, old value 0x22222222 replaced by 0xddddddddd. Commit 254a40d fixed three runner defects found by the rerun itself: a missing PyYAML on the Mac (the runner now has a stdlib fallback parser, keeping the suite self-contained), a --only flag the rerun instructions assumed but the runner did not have, and verdict accounting in which unselected gates counted as failures (they now report SKIPPED, and withdrawn F4 reports WITHDRAWN).

The Mac rerun on 2026-10-12 (log: results/run_20261012-230314.log) reported B4 PASS, B7 PASS, G1 PASS, G2 PASS, RESULT: ALL PASS. Combined with the 27 gates from the first run, the Mac stands at 31/31 active requirements, and the demo then ran clean there too (24/24 packets, TB_APP: ALL PASS). MAC_VALIDATION_REPORT.md was updated in commit cd7ee98 with the correction at the top and the original report kept below it for the record. Verilator 5.052 was verified fine for all four gates; no specific Verilator version is required. The GTKWave save file used for Figures 3 to 5 (demo/demo_trace.gtkw, commit 638679f) was added so the demo waveforms open preconfigured on the Mac.

7 · Human interventions and integrity notes

The owner intervened at the points where judgment, not execution, was required, and each intervention is in results/INTERVENTIONS.md (36 dated entries) or the manifest change list:

- **Contract approval and amendments.** He approved the contract before implementation, approved the B7 count reduction with its arithmetic in front of him, and withdrew F4 ('ignore SDF sim') when the tool limitation was proven rather than worked around.
- **Process rules that changed the work.** All project files under ~/work on his machine; one-time fine-grained GitHub tokens for pushes, never stored, revoked after use; no weakening of macOS security settings to get tools running; verdicts accepted only from logs and files, never from summary lines.
- **Corrections of the builder.** He rejected the first ARM diagnosis by directing a deeper investigation ('you managed to build this before'), which produced the evidence-based root cause in Section 6. A GTKwave Gatekeeper block during waveform viewing was resolved by launching from the terminal, without touching quarantine settings.
- **Recorded mistakes.** The ARM/Verilator diagnosis (Section 6.2) was written into a committed report before it was verified, and is corrected in the same file. During the Linux build, 36 intervention entries cover real bugs found and fixed along the way, including a UART RX overrun in firmware, a testbench reset race behind a B1 failure, and two rounds of event-rate mismatch in the preemption gate. None were hidden; each entry names the symptom, the cause, and the fix.

Integrity: the branch history is intact (99 commits), the default branch was never touched, and no acceptance criterion was weakened at any point. The two contract amendments changed a count to match measured reality and withdrew a gate the toolchain provably cannot run; both carry the owner's approval and their reasoning in the manifest itself.

8 · Known limitations

- **Zero-delay gate-level simulation.** F2 and F3 prove the synthesized netlist behaves like the RTL; they say nothing about timing. SDF-annotated timing simulation (F4) was withdrawn: Icarus 12.0 cannot simulate specify path delays, proven on both the packaged and source builds. The SDF infrastructure is retained in tree for a future tool.
- **Single-cycle core.** CPI = 1 by construction; characterization is functional, not a PPA signoff. No caches, no pipeline, no M/A/C extensions, machine mode only, wfi unimplemented (traps).
- **Simulator optimism.** Icarus is 4-state but optimistic on some paths; mitigated by defined reset values on all architectural state and by the reset-variation tests in A3, and cross-checked by running the long gates under Verilator.
- **Memory macros are models.** SRAM and boot ROM are behavioral models declared as black boxes in synthesis; a production flow would substitute compiled memories and rerun F1 to F3.
- **Spike on macOS.** No Spike build exists on the Mac; A1/A2 there compare against the committed golden signatures rather than regenerating them. The comparison is unaffected; regeneration is a Linux-side step.

9 · Reproduce it

On Ubuntu 24.04 (reference platform):

```
git clone https://github.com/mtmoreira/Muse-mcu
cd Muse-mcu && git checkout work/mcu-acceptance-v1
bash env/bootstrap.sh        # pinned, checksummed toolchains
make acceptance              # full suite; expect 31 PASS, F4 WITHDRAWN
```

On macOS (everything lands under ~/work):

```
cd ~/work/Muse-mcu
bash env/setup.sh            # toolchains under ~/work/toolchains
bash tools/run_tests.sh      # full suite
bash tools/run_tests.sh --only B4,B7,G1,G2   # targeted rerun
bash demo/run_demo.sh        # narrated demo
gtkwave results/raw/demo_trace.vcd demo/demo_trace.gtkw
```

Targeted reruns print honest verdicts: gates outside the selection report SKIPPED, F4 reports WITHDRAWN, and RESULT: ALL PASS is printed only when every selected gate passed.

Appendix · Key evidence index

Question	File
What is required?	acceptance/manifest.yaml (v1.0.3, with change list)
Did each gate pass?	results/ACCEPTANCE_STATUS.md; results/*_gate.json; results/raw/ logs
Final Linux verdict	results/ACCEPTANCE_REPORT.md (31 PASS, re-verified 2026-10-09)
Mac verdict	MAC_VALIDATION_REPORT.md; results/run_20261012-230314.log
Synthesis numbers	results/f1_synth.json (cells, area, netlist SHA-256)
Demo transcript	results/demo_transcript.txt
Demo waveforms	results/raw/demo_trace.vcd (50 MB); view: demo/demo_trace.gtkw
What went wrong when	results/INTERVENTIONS.md (36 entries)
Architecture spec	doc/architecture.md; figures doc/fig_arch.png, doc/fig_trace.png
Reuse pins	third_party/PINS.md (per-file SHA-256)

Report ends. The repository is the evidence; this document is a map to it.